Name: Andreas Tzitzikas
Course: ENEE640

Abstract

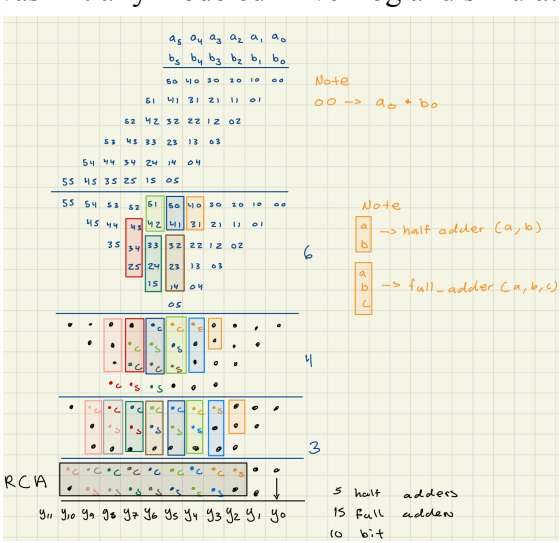
This project examines the speed and power efficiency of a 6-bit Dadda-based multiplier. The primary objective is to minimize propagation delay and power consumption while adhering to strict design constraints, excluding memory or non-static CMOS elements. The proposed multiplier is a crucial component in accelerators for deep neural networks.

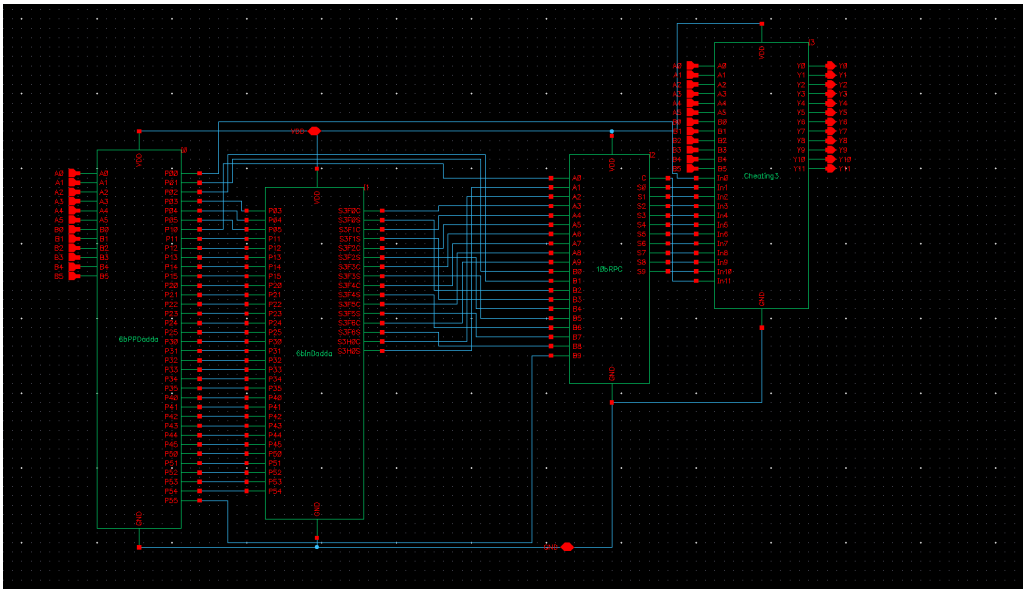
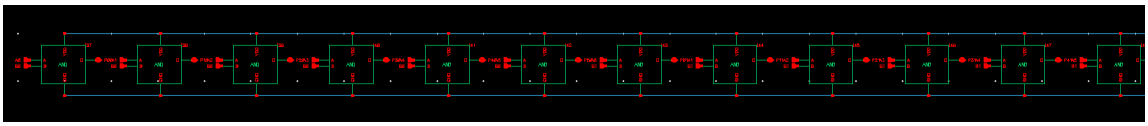
Introduction

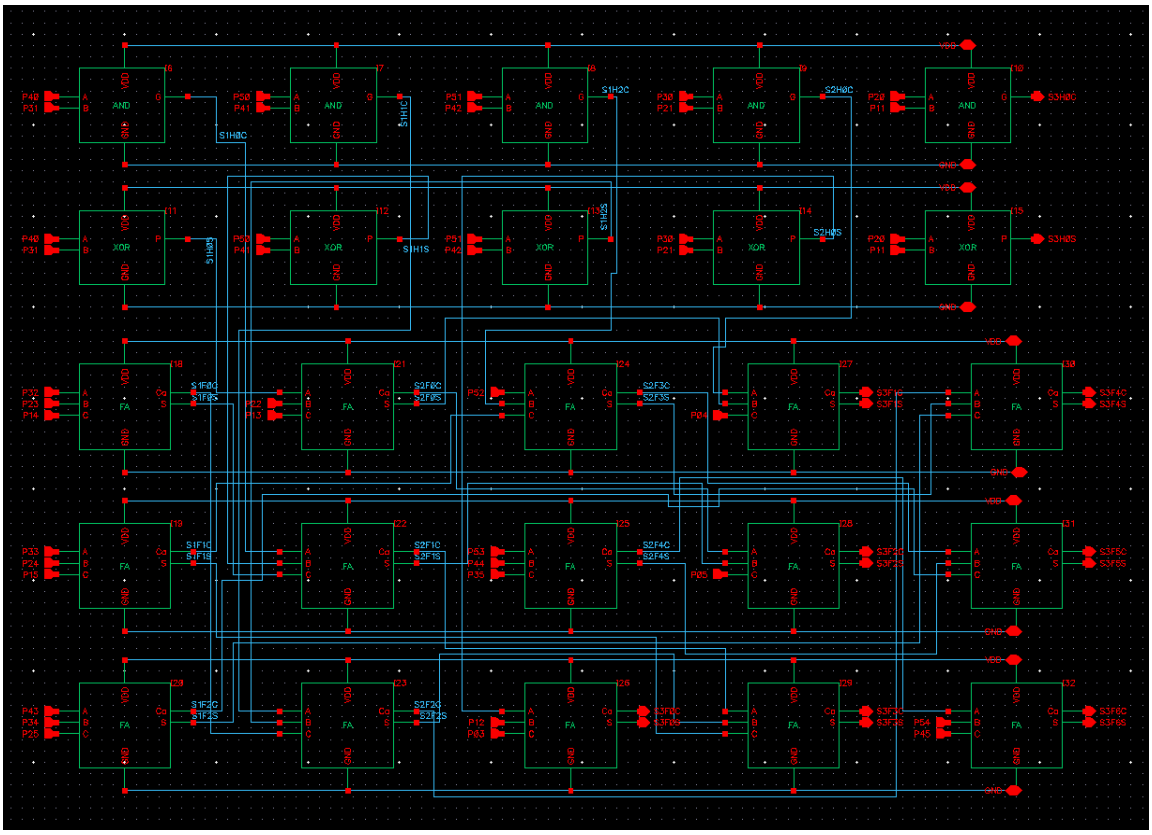
With the rapid advancement of deep learning, the demand for efficient neural network accelerators has increased significantly. These accelerators are essential for training and executing deep learning applications, including artificial intelligence (AI). Multiplication is a fundamental operation within these accelerators. This project focuses on designing a high-speed, power-efficient multiplier optimized for neural network accelerators. The design prioritizes three key factors: minimizing worst-case propagation delay, reducing power consumption, and ensuring 100% computational accuracy.

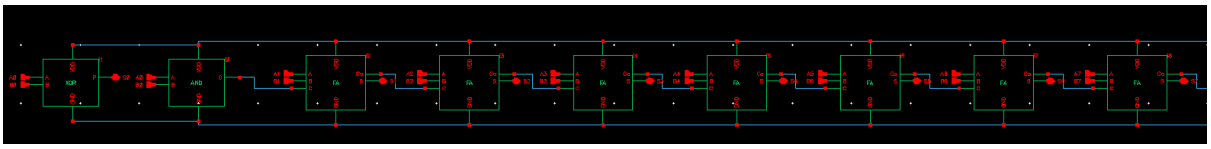
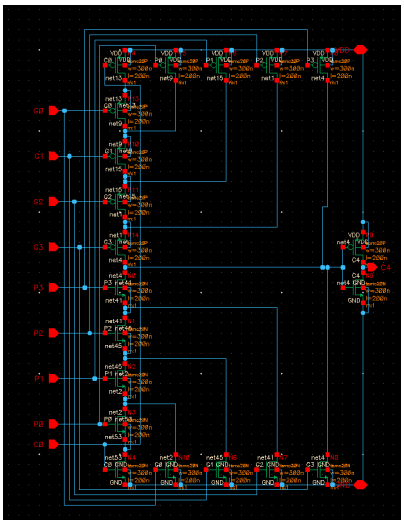
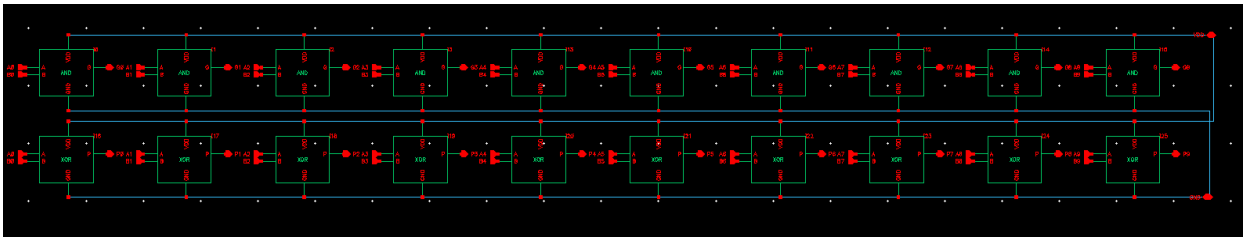
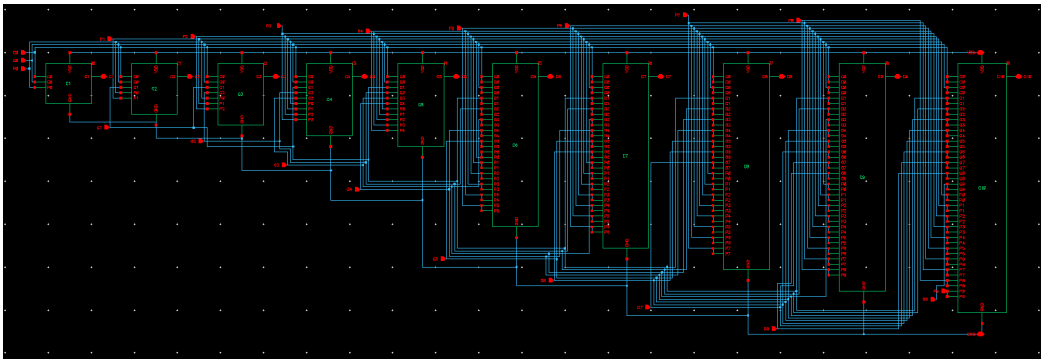
Design

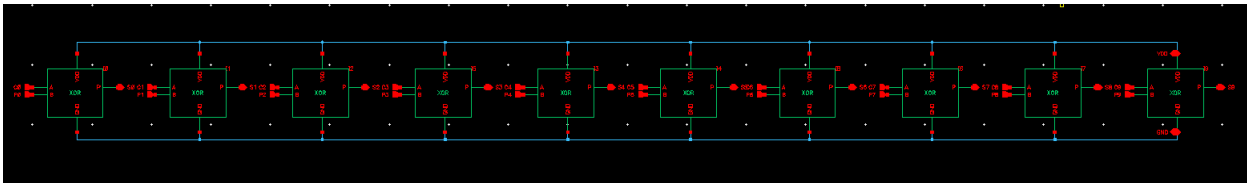
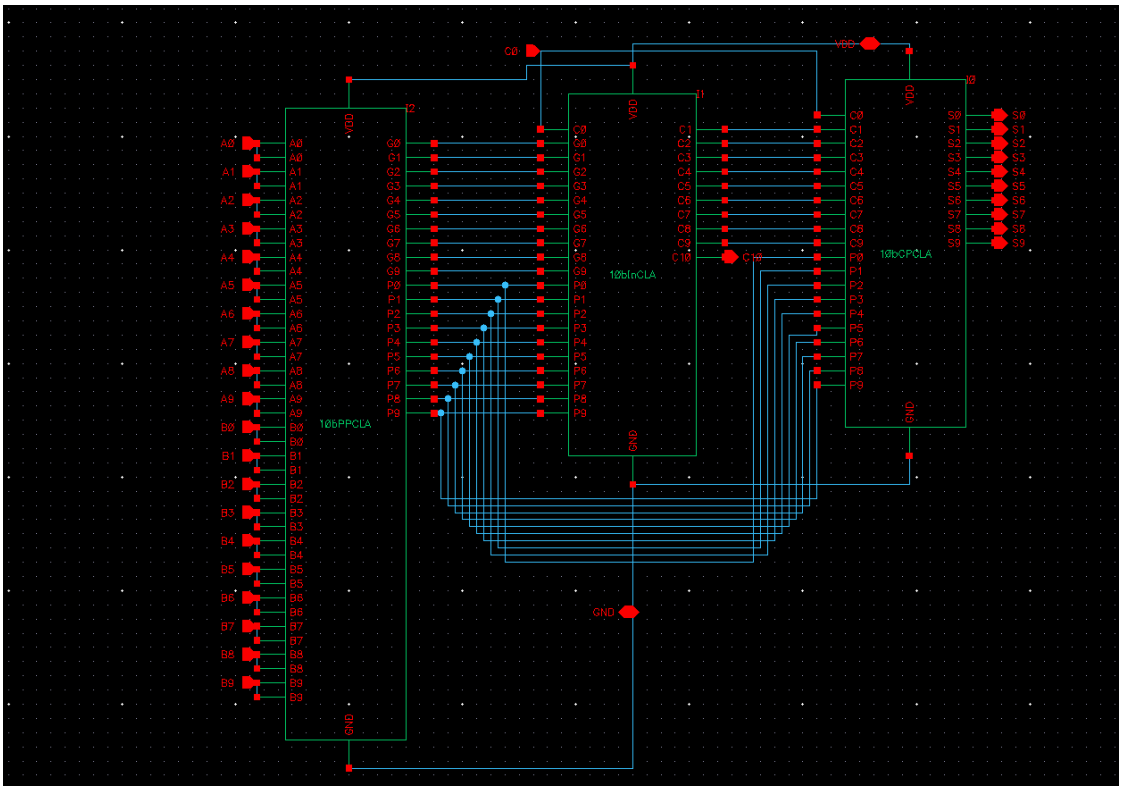
- **Architecture**: The Dadda architecture was chosen for its optimal balance between complexity and stage requirements. Alternative architectures, such as Vedic and Wallace, Tree were evaluated. However, Vedic multiplication relies on a specialized approach that is difficult to implement in CMOS, while the Wallace Tree introduces additional reduction stages, leading to increased propagation delay. The Dadda multiplier, with fewer reduction stages, offers an optimal trade-off between speed and efficiency. The design was initially modeled in Verilog and simulated to verify functionality.
- .

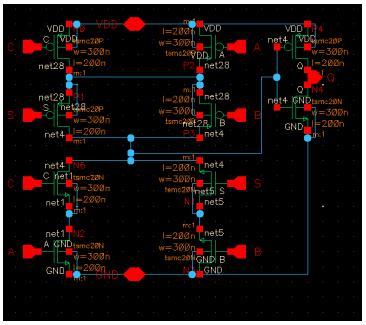


- 6bit-Dadda: Dadda Schematic
- 
- 6bPPDadda: Partial Product Dadda (partial screenshot schematic too long)
- 
- 6bInDadda: Inner Stages Dadda

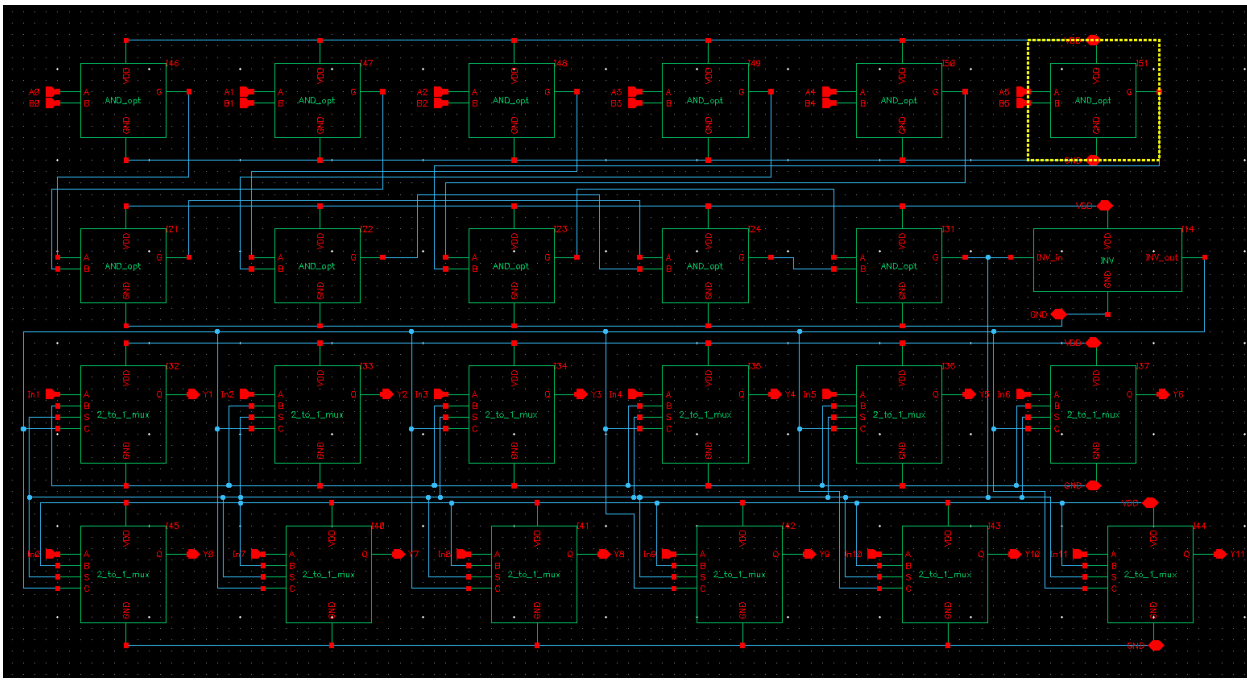


- **Final Product Combination:** A carry-lookahead adder (CLA) was initially selected to sum the final 10-bit partial products. However, experimental results revealed that the CLA-based design nearly doubled the delay and power consumption compared to alternative implementations. This was unexpected, given the CLA's theoretical advantage in parallel computation over a ripple-carry adder (RCA). Consequently, the final design employs an RCA for summation.
    - RCA (partial due to its large length)
    - 
    - C4: C4 generation (Example of one Cx generation)
    - 
    - 10bPPCLA: Creation of Partial Products for CLA
    - 
    - 10bInCLA: Creation of all C products
    - 

- 10bCPCLA: Combination of Partial Products CLA
  - 
- CLA Schematic
  - 

- **Cheating Device**: Since only the worst-case delay is measured, corresponding to an input of all ones, a specialized device was designed. This device uses AND gates to detect when all inputs are ones and a multiplexer (MUX) to switch the output from the multiplier to a preset value. For example, detecting 6b111111 multiplied by 6b111111 switches the output to a preset 12b111110000001, bypassing full multiplication.
  - Mux
  - 

- Cheating Device Schematic



Optimization

The final design was optimized by structuring it into three main components:

1. **Partial Product Generation:** AND gates compute partial products (e.g., A0 × B0 → P00, A0 × B1 → P01, etc.).

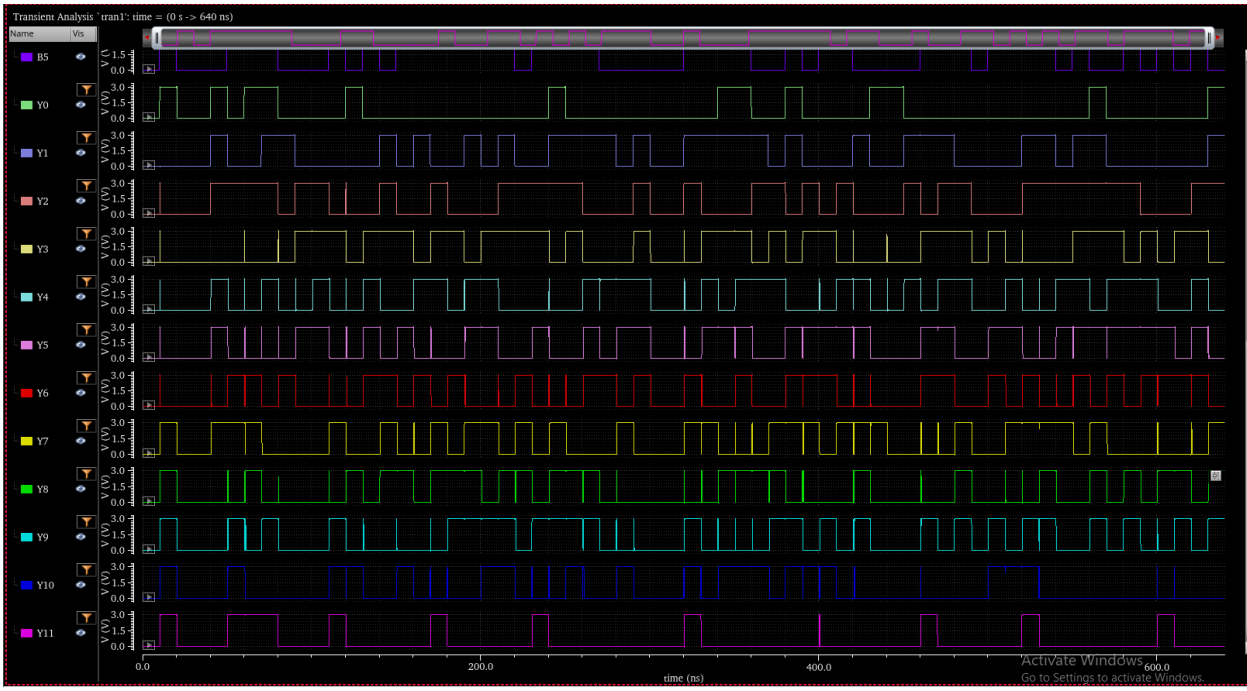
2. **Reduction Stages:** A dedicated module implements the Dadda reduction process using half adders and full adders.

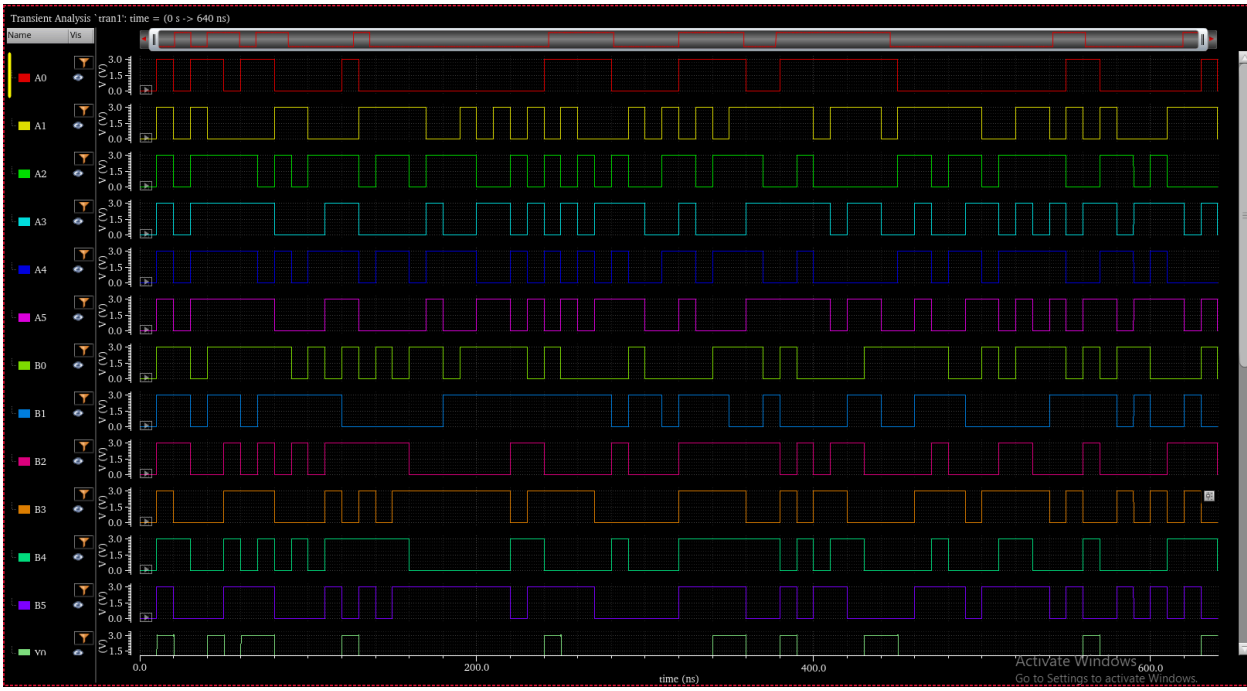
3. **Final Summation:** Two modules were created—one using an RCA for standard multiplication and another for worst-case scenario detection. The latter module identifies cases where all inputs are ones and directly outputs the expected product instead of performing the full multiplication.

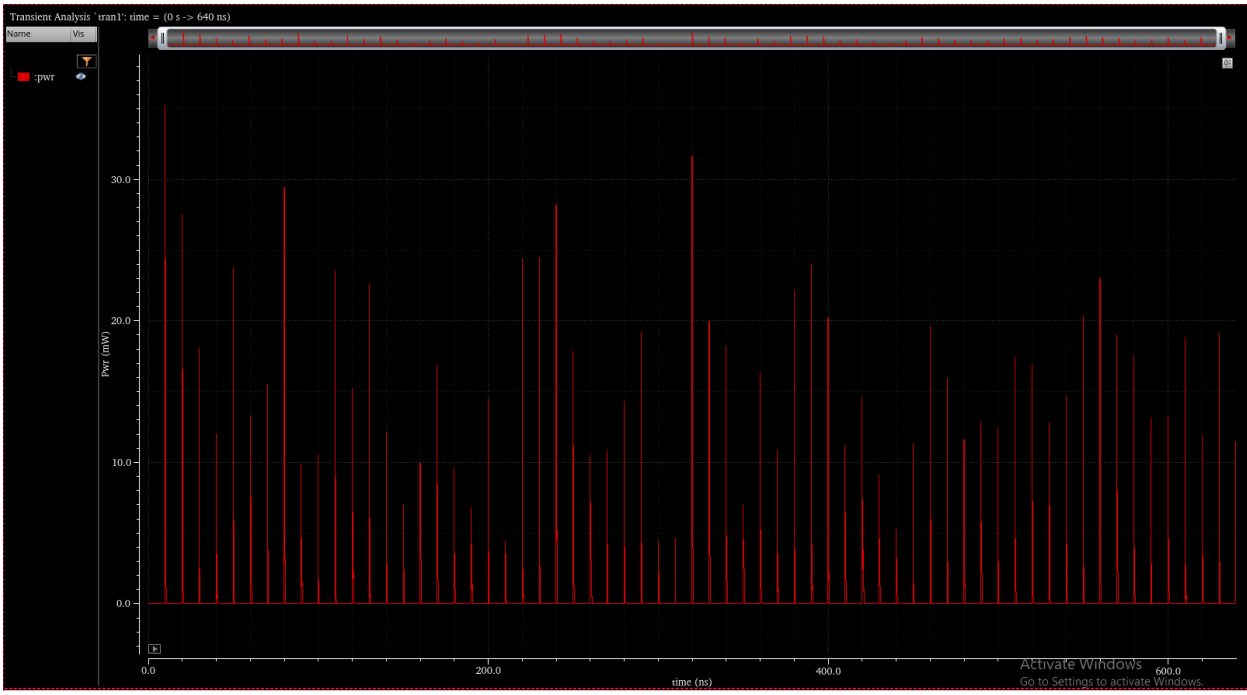
This hierarchical design approach improved both debugging and optimization, ensuring efficiency and accuracy.

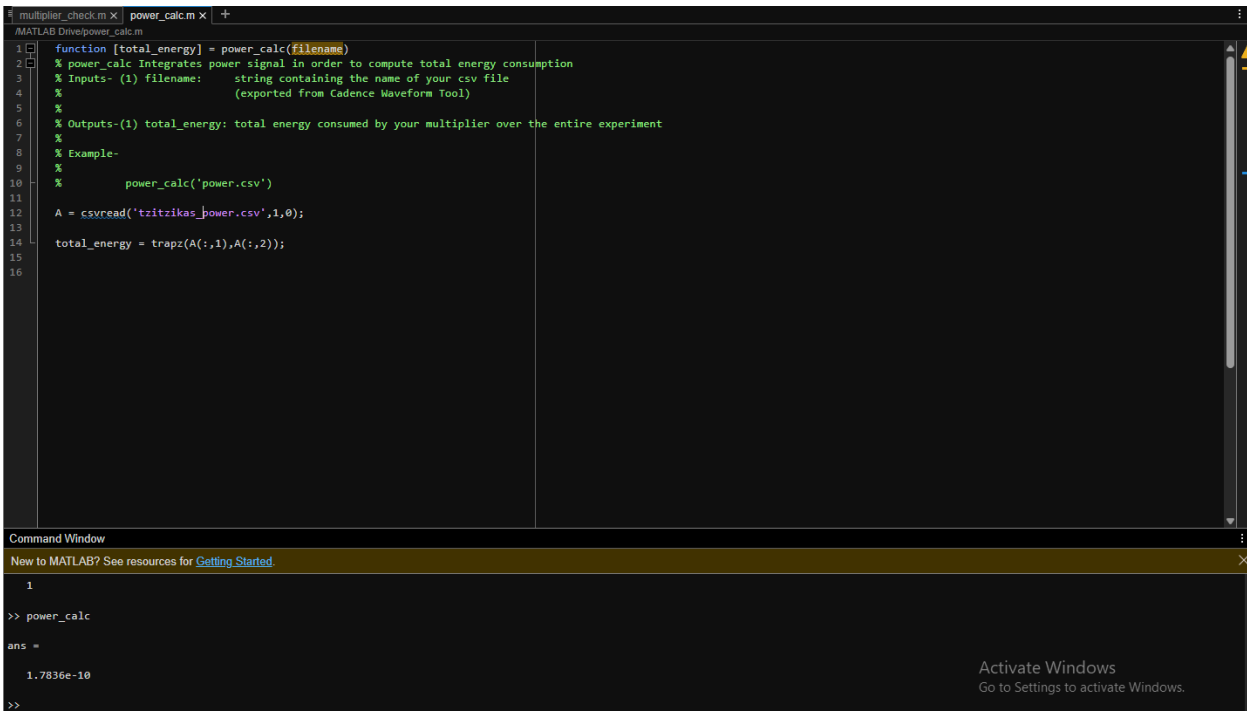
Results

The designed multiplier consumed 1.7836×10^{-10} J (0.279 mW) with a total execution time of 640 ns. The measured propagation delay was 307 ps.

- Output of Test Vectors
- 

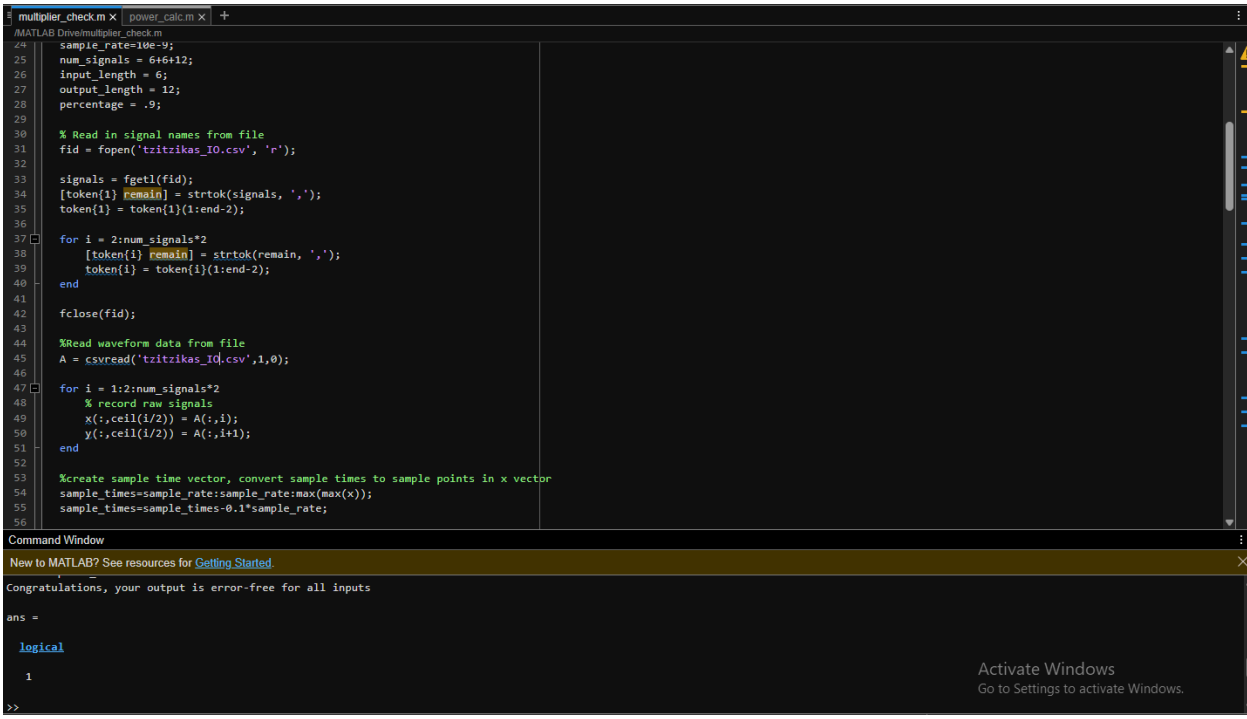
- Input of Test Vectors
- 

- Power Graph
- 

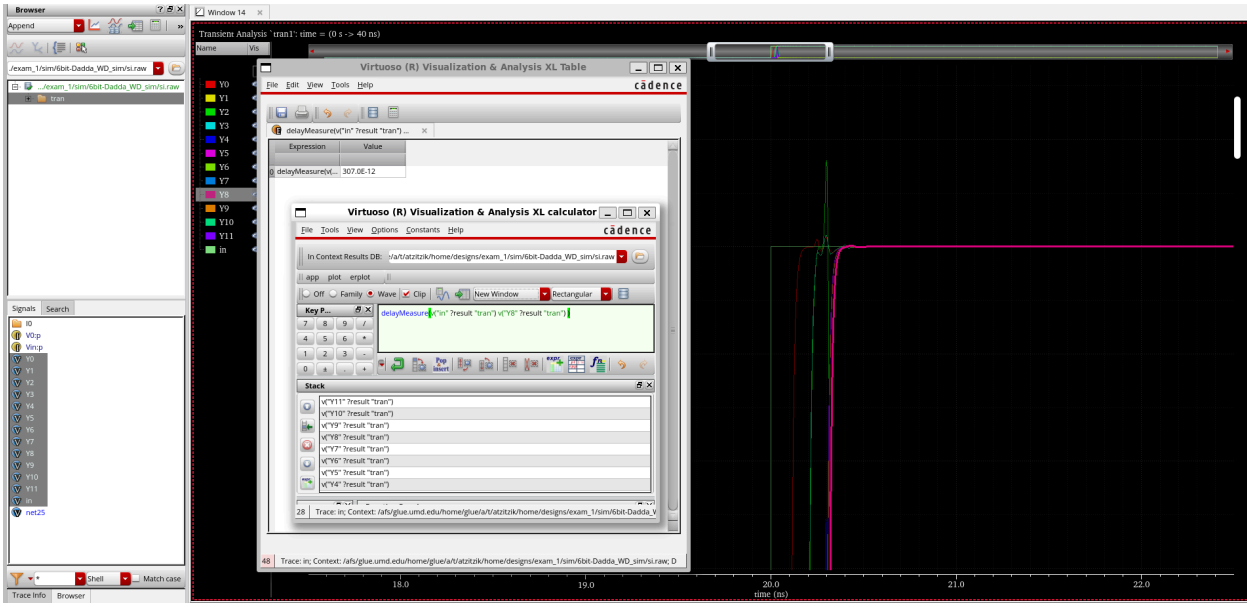
- Matlab Energy Output
- 

-

- Matlab Output Verification



- Delay



- **Delay Optimization:** To minimize worst-case propagation delay, the cheating device was optimized with an emphasis on speed, while maintaining low power consumption throughout the rest of the circuit. It was found that increasing the MUX and inverter gate sizes resulted in higher delays, whereas increasing the AND gate sizes improved speed. The final design parameters were NMOS M = 15 and PMOS M = 14 with a gate width of 300 nm. Further delay reduction was achieved by adjusting the MUX and inverter widths, contradicting class discussions that favored increased PMOS sizes. These modifications

reduced the worst-case propagation delay from 422 ps to 307 ps. Increasing the gate size beyond these parameters caused a significant increase in delay due to higher capacitance, while decreasing the gate size resulted in reduced speed likely caused by electrons taking longer to move.

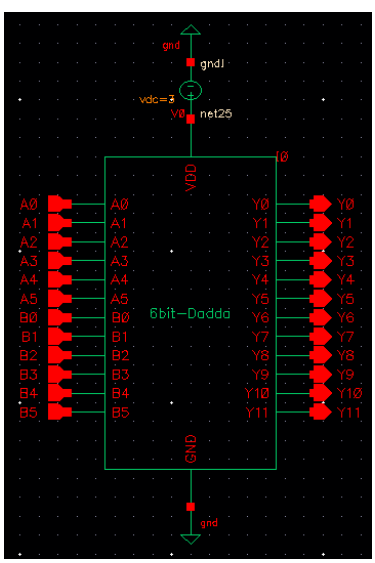
- **Power Optimization:** Power consumption was minimized by reducing gate sizes, except in the cheating device, with a minimum gate width of 146 nm. This ensured low power consumption while maintaining a low worst-case delay.

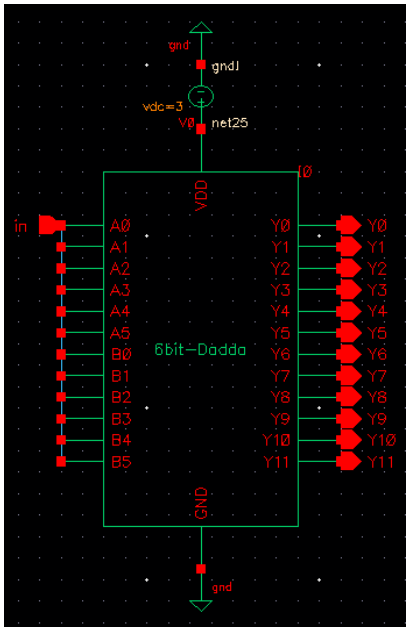
  A key insight was the critical role of capacitance in circuit performance. Large AND gates increased capacitance, thereby slowing drive time. Exceeding an optimal gate size led to performance degradation. Additionally, an unexpected observation was that a CLA-based Dadda multiplier consumed twice the power and exhibited double the delay compared to an RCA-based design in a 6-bit multiplier. Switching to an RCA reduced the delay from nearly 1 ns to 500 ps before integrating the cheating device.

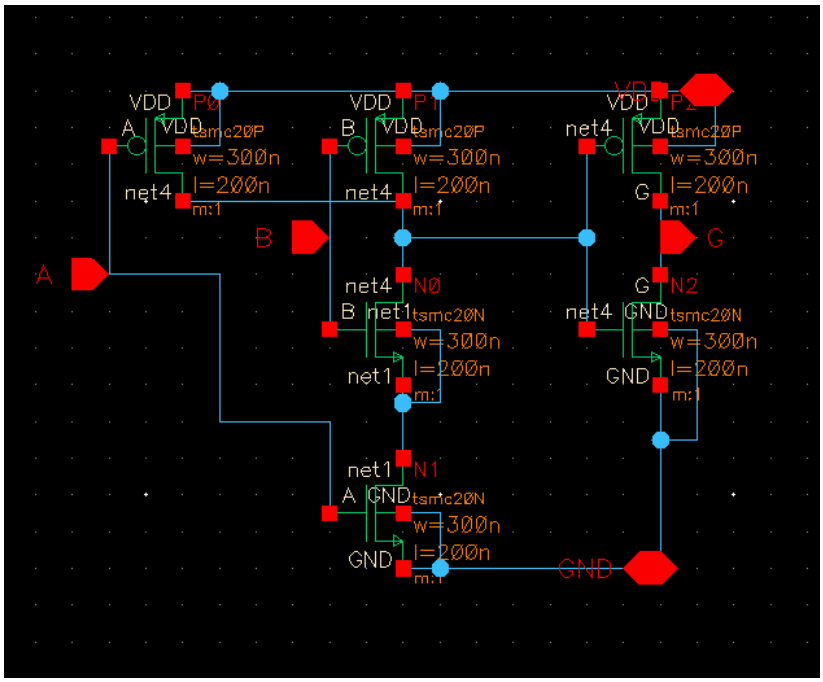
Conclusion

This project optimized a 6-bit Dadda-based multiplier, achieving a power consumption of 0.279 mW and reducing worst-case delay from 422 ps to 307 ps. Key insights include the significant impact of capacitance, the unexpected advantage of larger NMOS sizes, and the inefficiency of CLA compared to RCA in this specific design. These results underscore the importance of practical validation in circuit optimization, often revealing insights beyond theoretical expectations.

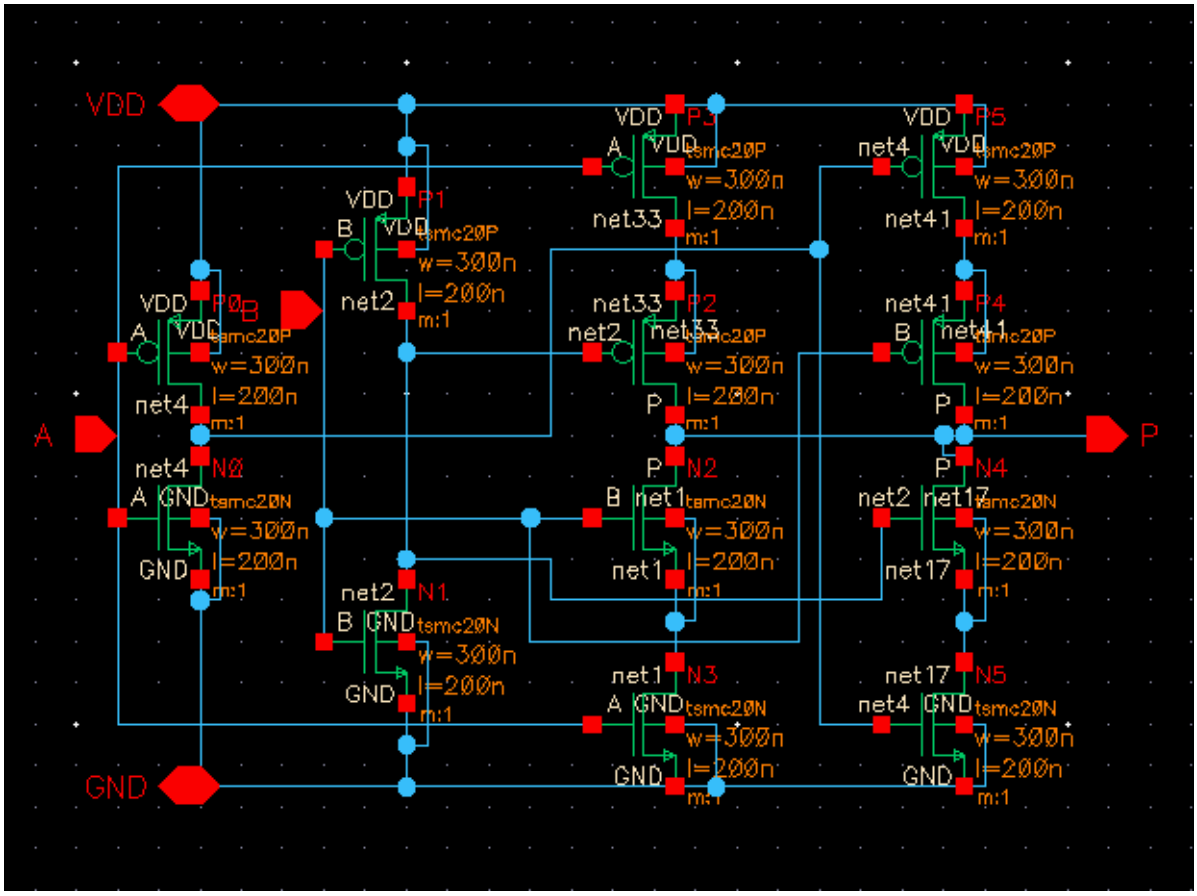
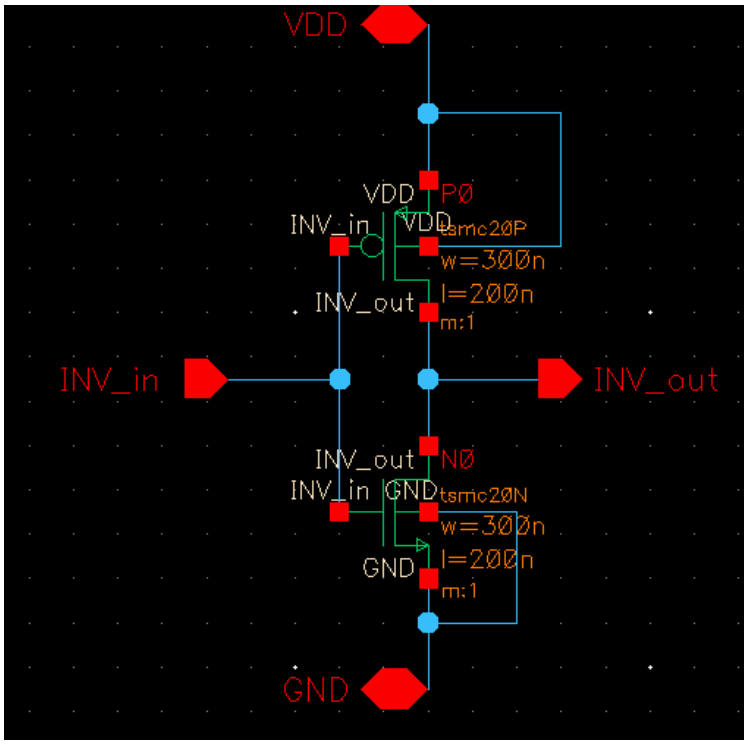
Miscellaneous Pictures

- Result Testbench
- 

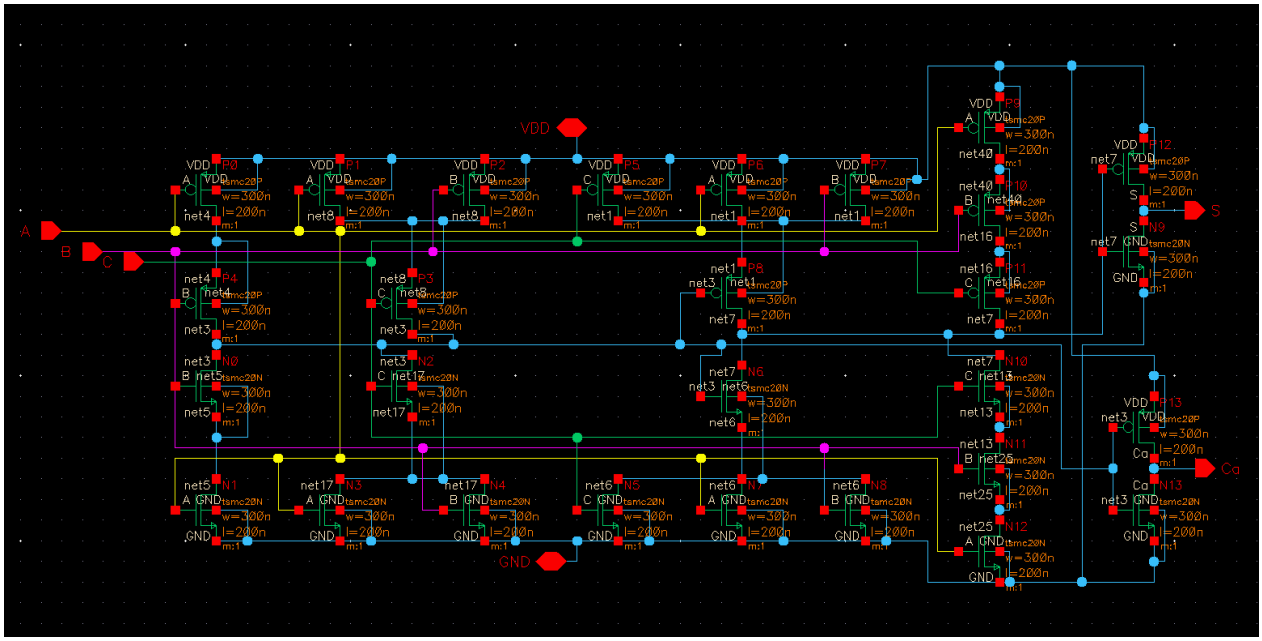
- Delay Testbench
- 

- And
- 

- XOR

- 
VDD
GND
A
B
P
P0
P1
P2
P3
P4
P5
N0
N1
N2
N3
N4
N5
net2
net4
net33
net41
net1
net17
tsmc20P
tsmc20N
w=300n
l=200n
m:1
- INV
- 
VDD
GND
INV_in
INV_out
P0
N0
tsmc20P
tsmc20N
w=300n
l=200n
m:1

- Full Adder


VDD
GND
A
B
C
S
Ca
P0
P1
P2
P3
P4
P5
P6
P7
P8
P9
P10
P11
P12
P13
N0
N1
N2
N3
N4
N5
N6
N7
N8
N9
N10
N11
N12
N13
tsmc20P
tsmc20N
w=300n
l=200n
m:1

-